Actividad A01.8. Procesos paso a paso

Programación de servicios y procesos (0490) · 2.º DAM · UD 1. Procesos · Curso 2026-2027

> **No cuenta para la nota.** Son diez ejercicios para entender, poco a poco, el código de un padre que lanza, termina y se comunica con sus hijos, y para preparar la prueba de RA1. Hazlos tú: si te los hace una IA, no te sirven para la prueba. Si te atascas, mira el ejemplo de la unidad que se indica o pregunta.

1. Qué tienes que hacer

Cada ejercicio es un programa casi terminado: un **padre** que lanza uno o varios **hijos**. Solo tienes que escribir unas líneas en el sitio que marca un comentario `// TODO`. Los primeros piden una o dos líneas y cada ejercicio añade algo nuevo, hasta el último, en el que el padre maneja varios hijos a la vez.

N.º	Qué practicas	Hijo	Líneas	Modelo
1	Lanzar un hijo y esperar a que termine	`Saludo`	2	6.3
2	Pasarle argumentos	`Saludo`	3	7.1
3	El comando que lanza un programa Java	`Saludo`	4	6.3
4	Terminar un hijo	`Dormilon`	4	6.4
5	Leer la salida estándar del hijo	`Division`	3	7.2a
6	Escribir en la entrada estándar del hijo	`Mayusculas`	6	7.2b
7	Redirigir los flujos del hijo a ficheros	`Mayusculas`	5	7.3
8	El código de salida y la salida de error	`Division`	9	8.1a
9	Una tubería de tres procesos	`GeneradorNumeros`, `Cuadrado`, `Sumar`	9	7.5
10	Varios hijos a la vez, con tiempo límite	`Dormilon`	15	8.1b, 9.1

Hazlos **en orden**: cada uno da por sabido lo de los anteriores. Después de cada ejercicio hay unas preguntas (**Piensa**) que te piden cambiar algo y ver qué pasa. Son lo más importante: cuando sepas responderlas, habrás entendido el código, no solo copiado.

2. Qué se practica

RA1. Desarrolla aplicaciones compuestas por varios procesos reconociendo y aplicando principios de programación paralela.

CE	Criterio de evaluación
CE1.e	Se han utilizado clases para programar aplicaciones que crean subprocesos.
CE1.f	Se han utilizado mecanismos para compartir información con los subprocesos iniciados.
CE1.g	Se han utilizado mecanismos para sincronizar y obtener el valor devuelto por los subprocesos iniciados.
CE1.h	Se han desarrollado aplicaciones que gestionen y utilicen procesos para la ejecución de varias tareas en paralelo.

3. Antes de empezar

Material de partida. En el Aula Virtual, `partida-A01-8.zip`. Descomprímelo y ábrelo en IntelliJ IDEA con *File* > *Open*, eligiendo la carpeta `partida-A01-8`. Si te pide el JDK, elige el 21.

- Cada ejercicio es una clase: `Ej01Lanzar`, `Ej02Argumentos`... Se ejecuta desde el desplegable de configuraciones (arriba a la derecha) o con el botón derecho sobre el fichero, *Run*.
- Todas las clases están en la misma carpeta, sin paquete. Por eso los hijos se compilan junto con los padres y el padre los encuentra con su propio *classpath*.
- Los **hijos ya están hechos** y no hay que cambiarlos. Lo que necesitas saber de ellos está en el apartado 4.

Cómo es un hueco. El comentario de encima del método dice qué tiene que hacer, y el `TODO`, cuántas líneas se esperan:

```
/** Lanza el proceso que describe pb y devuelve el Process del hijo. */
static Process lanzar(ProcessBuilder pb) throws IOException {
    // TODO (1 línea): lanza el proceso de pb y devuelve su Process
    return null;  // provisional, para que compile: bórralo
}
```

- El `return` provisional solo está para que el proyecto compile: **bórralo** al escribir tu código.
- El número de líneas es el de la solución, contando las llaves de cierre. Es orientativo: si te salen muchas más, vuelve a leer la pista.
- No cambies el resto del código, salvo cuando te lo pida una pregunta de *Piensa* (y después deshaz el cambio).
- Si ejecutas un ejercicio sin hacer, falla (`NullPointerException`, `IOException`...) o, en el ejercicio 6, se queda colgado. Es normal. Un programa colgado se para con el botón *Stop*, el cuadrado rojo.
- Compara lo que te sale con la **salida esperada**. Los PID y las rutas serán otros en tu equipo, y en algunos ejercicios el orden de algunas líneas puede cambiar: la pregunta de *Piensa* explica por qué.

4. Los programas hijo

Este es el **contrato** de cada hijo: cómo se usa, qué hace con sus flujos y qué código de salida devuelve. Es lo único que el padre necesita saber de él.

Hijo	Uso	Qué hace	Código de salida
`Saludo`	`java Saludo [nombre]`	Escribe dos líneas: saluda al nombre (si no lo recibe, a «desconocido») y dice su PID y el de su padre	0
`Dormilon`	`java Dormilon <segundos>`	Duerme esos segundos y escribe en la salida estándar cuántos ha dormido. En la salida de error avisa al dormirse y al despertarse	0; 1 si falta el argumento
`Division`	`java Division <dividendo> <divisor>`	Escribe en la salida estándar el resultado de la división entera. Si algo va mal, explica el error en la salida de error y no escribe nada en la estándar	0 si va bien; 1 si no hay dos argumentos; 2 si no son números enteros; 3 si el divisor es 0
`Mayusculas`	`java Mayusculas`	Lee frases de la entrada estándar y las escribe en mayúsculas en la salida estándar. Las frases en blanco no las copia: avisa en la salida de error. Termina cuando se acaba la entrada	0
`GeneradorNumeros`	`java GeneradorNumeros <desde> <hasta>`	Escribe en la salida estándar los números del intervalo, uno por línea (el del ejemplo 7.5)	0; 1 si faltan argumentos

Hijo	Uso	Qué hace	Código de salida
`Cuadrado`	`java Cuadrado`	Lee números de la entrada estándar, uno por línea, y escribe el cuadrado de cada uno. Termina cuando se acaba la entrada	0
`Sumar`	`java Sumar`	Lee números de la entrada estándar, uno por línea, y cuando se acaba la entrada escribe su suma	0

5. Los ejercicios

Ejercicio 1. Lanzar un hijo y esperar a que termine

`Ej01Lanzar` · Hijo: `Saludo` · Completas `lanzar` y `esperar` (1 línea cada uno) · Modelo: ejemplo 6.3

El `main` ya prepara el `ProcessBuilder` del hijo. Faltan los dos pasos que vas a dar siempre: **lanzar** al hijo y **esperar** a que termine.

Pista. `start()` crea el proceso y devuelve su `Process`. `waitFor()` bloquea al padre hasta que el hijo termina y devuelve su código de salida.

Salida esperada:

```
Padre: soy el proceso 4120
Padre: he lanzado un hijo con PID 4133
Hijo: hola, desconocido. He recibido 0 argumento(s).
Hijo: soy el proceso 4133 y mi padre es el 4120.
Padre: el hijo ha terminado con código 0
```

Piensa.

1. Busca en la salida el PID del padre y el del hijo. ¿Qué PID dice el hijo que tiene su padre?
2. Quita la línea `pb.inheritIO();` y ejecuta. ¿Se ve el saludo del hijo? ¿Dónde ha ido a parar?

Ejercicio 2. Pasar argumentos a un hijo

`Ej02Argumentos` · Hijo: `Saludo` · Completas `lanzarSaludo` (3 líneas) · Modelo: ejemplo 7.1

Ahora el `ProcessBuilder` lo creas tú. El `main` lanza tres saludos, uno detrás de otro. Cada hijo tiene que recibir el nombre como argumento y escribir en la consola del padre.

Pista. Los argumentos van detrás del nombre de la clase: `comandoJava("Saludo", ...)`. Para que el hijo escriba en la consola del padre, lo mismo que en el ejercicio 1.

Salida esperada:

```
Hijo: hola, Ana. He recibido 1 argumento(s).
Hijo: soy el proceso 4201 y mi padre es el 4199.
Hijo: hola, Luis. He recibido 1 argumento(s).
Hijo: soy el proceso 4202 y mi padre es el 4199.
Hijo: hola, Juan Carlos. He recibido 1 argumento(s).
Hijo: soy el proceso 4203 y mi padre es el 4199.
Padre: he saludado a 3 personas
```

Piensa.

1. «Juan Carlos» lleva un espacio, y el hijo dice que ha recibido un solo argumento. ¿Qué recibe si en `lanzarSaludo` pones `comandoJava("Saludo", "Juan", "Carlos")`?
2. Quita del `main` la línea `hijo.waitFor();` y ejecuta varias veces. ¿En qué orden salen los mensajes? ¿Por qué el padre puede despedirse antes de que saluden sus hijos?

Ejercicio 3. El comando que lanza un programa Java

`Ej03Comando` · Hijo: `Saludo` · Completas 4 líneas de `comandoJava` · Modelo: ejemplo 6.3

Hasta ahora `comandoJava` venía hecho. Lo que hace es construir el comando que escribirías en la terminal, `java -cp <classpath> <Clase> <argumentos...>`, con una cadena por cada parte. El método ya crea la lista y añade los argumentos al final; tú añades lo que va antes.

Pista. La carpeta de la JVM que ejecuta al padre está en la propiedad `java.home`, y el programa `java`, en su subcarpeta `bin`. El *classpath* del padre está en `java.class.path`. Las dos se leen con `System.getProperty(...)`.

Salida esperada (las rutas dependen de tu equipo):

```
Padre: voy a lanzar este comando, parte a parte:
  <carpeta del JDK>\bin\java
  -cp
  <carpeta del proyecto>\out\production\partida-A01-8
  Saludo
  Marta
Hijo: hola, Marta. He recibido 1 argumento(s).
Hijo: soy el proceso 4310 y mi padre es el 4302.
Padre: el hijo ha terminado con código 0
```

Piensa.

1. Ejecútalo antes de completar el método. ¿Qué error da? ¿Por qué se queja de «Marta»?
2. Con el método hecho, cambia `"Saludo"` por `"Saludos"`, una clase que no existe. ¿Lanza `start()` una excepción? ¿Quién escribe el error, el padre o el hijo? ¿Con qué código termina el hijo?

Ejercicio 4. Terminar un hijo

`Ej04Terminar` · Hijo: `Dormilon` · Completas `terminar` (4 líneas) · Modelo: ejemplo 6.4

El hijo va a dormir 60 segundos, pero el padre solo le deja 3. Después tiene que terminarlo.

Pista. `isAlive()` dice si el hijo sigue en marcha. `destroy()` le pide que termine, pero **no espera** a que lo haga.

Salida esperada:

```
Padre: he lanzado un hijo con PID 4410
Hijo 4410: me duermo 60 s
Padre: ¿sigue vivo? true
Padre: ¿sigue vivo? false
Padre: el hijo ha terminado con código 143
```

El último número depende del sistema operativo: 143 en Linux y macOS, y otro distinto de 0 en Windows. Anota el que te sale.

Piensa.

1. ¿Por qué no sale nunca el mensaje `me despierto`?
2. Cambia `destroy()` por `destroyForcibly()`. ¿Cambia el código de salida?
3. En `terminar`, cambia `hijo.waitFor()` por `hijo.exitValue()`. ¿Qué pasa? ¿Por qué hay que esperar al hijo después de `destroy()`?

Ejercicio 5. Leer la salida estándar del hijo

`Ej05LeerSalida` · Hijo: `Division` · Completas `leerResultado` (3 líneas) · Modelo: ejemplo 7.2a

El hijo hace la división y escribe el resultado en su salida estándar. El padre tiene que leerlo y convertirlo en número para poder calcular con él.

Pista. Lo que el hijo escribe, el padre lo lee: la salida estándar del hijo es una **entrada** para el padre, `hijo.inputReader()`.

Salida esperada:

```
Padre: el hijo dice que 84 / 2 = 42
Padre: y el doble es 84
Padre: el hijo ha terminado con código 0
```

Piensa.

1. Aquí no hay `inheritIO()`. Añade `.inheritIO()` antes de `.start()` en el `main`. ¿Qué pasa? ¿Por qué?
2. Llama dos veces seguidas a `readLine()` y convierte en número lo que devuelve la segunda. ¿Qué devuelve? ¿Cuándo devuelve `readLine()` ese valor?

Ejercicio 6. Escribir en la entrada estándar del hijo

`Ej06EscribirEntrada` · Hijo: `Mayusculas` · Completas `enviar` (6 líneas) · Modelo: ejemplo 7.2b

Ahora es al revés: el padre escribe y el hijo lee. El padre envía unas frases al hijo, que se las devuelve en mayúsculas. El `main` ya lee lo que devuelve el hijo.

Antes de empezar, ejecútalo sin completar `enviar`: se queda colgado. Páralo con *Stop*. La primera pregunta de *Piensa* es por qué.

Pista. La entrada estándar del hijo es una **salida** para el padre, `hijo.outputWriter()`. Detrás de cada frase, `newLine()`. Abre el escritor en un `try (...)`, como el lector del ejercicio 5: al terminar, lo cierra.

Salida esperada:

```
Padre: el hijo me devuelve:
Hijo: frase en blanco, no la copio
  HOLA
  ESTO LO ESCRIBE EL PADRE
  Y EL HIJO LO PASA A MAYUSCULAS
Padre: el hijo ha terminado con código 0
```

Piensa.

1. ¿Por qué se quedaba colgado sin `enviar`? ¿Qué estaba esperando el padre? ¿Y el hijo?
2. Escribe las frases sin `try`, con `BufferedWriter entrada = hijo.outputWriter();`, y sin cerrar el escritor. ¿Qué pasa? ¿Para qué sirve cerrarlo?
3. El aviso de la frase en blanco no pasa por el padre, y puede salir antes o después de `Padre: el hijo me devuelve:`. ¿Por qué?

Ejercicio 7. Redirigir los flujos del hijo a ficheros

`Ej07Redirecciones` · Hijo: `Mayusculas` · Completas `lanzarRedirigido` (5 líneas) · Modelo: ejemplo 7.3

El mismo hijo, pero esta vez el padre no le escribe ni le lee nada: conecta sus tres flujos con ficheros. La entrada sale de `frases.txt`, la salida va a `mayusculas.txt` (si ya existe, se sustituye) y la salida de error se **añade** al final de `avisos.log`. El `main` crea `frases.txt` y, cuando el hijo termina, muestra los otros dos ficheros.

Pista. Un `redirect...` del `ProcessBuilder` por cada flujo. Para añadir al final de un fichero en lugar de sustituirlo, `ProcessBuilder.Redirect.appendTo(...)`.

Salida esperada:

```
Padre: el hijo ha terminado con código 0
--- mayusculas.txt ---
PRIMERA FRASE
SEGUNDA FRASE
TERCERA Y ULTIMA
--- avisos.log ---
Hijo: frase en blanco, no la copio
Hijo: frase en blanco, no la copio
```

Piensa.

1. Ejecútalo otra vez. ¿Cuántas líneas tiene ahora `avisos.log`? ¿Y `mayusculas.txt`? ¿Por qué?
2. ¿En qué carpeta aparecen los tres ficheros? ¿Por qué ahí?
3. En el ejercicio 6, el padre tenía que cerrar el escritor para que el hijo terminara. ¿Cómo sabe aquí el hijo que se han acabado las frases?

Ejercicio 8. El código de salida y la salida de error

`Ej08CodigoSalida` · Hijo: `Division` · Completas `informe` (9 líneas) · Modelo: ejemplo 8.1a

El padre lanza cuatro divisiones y tres van mal. El **código de salida** dice cómo le ha ido a cada hijo: si es 0, su resultado está en la salida estándar; si no, el motivo está en la salida de error. El comentario de `informe` dice qué texto tiene que devolver en cada caso.

Pista. Primero, `waitFor()`, que te da el código. La salida de error del hijo se lee con `hijo.errorReader()`, igual que la estándar.

Salida esperada:

```
Division 84 2 -> resultado 42
Division 7 0 -> error 3: No se puede dividir entre 0
Division 7 -> error 1: Uso: java Division <dividendo> <divisor>
Division siete 2 -> error 2: Los argumentos tienen que ser números enteros: siete 2
```

Piensa.

1. Cuando el código no es 0, ¿qué hay en la salida estándar del hijo? Compruébalo: lee siempre la salida estándar, vaya bien o mal.
2. ¿Quién decide qué significa cada código de salida, el padre o el hijo? ¿Dónde se mira?

Ejercicio 9. Una tubería de tres procesos

`Ej09Tuberia` · Hijos: `GeneradorNumeros`, `Cuadrado` y `Sumar` · Completas `lanzarTuberia` (5 líneas) y `leerUltimo` (4 líneas) · Modelo: ejemplo 7.5

Tres hijos encadenados, como `GeneradorNumeros 1 10 | Cuadrado | Sumar` en la terminal: la salida de cada uno es la entrada del siguiente, y el padre solo lee la del último. Entre los hijos el padre no escribe ni lee nada: los conecta el sistema operativo.

Pista. Prepara un `ProcessBuilder` por etapa, en una lista y en orden. `ProcessBuilder.startPipeline(...)` los lanza conectados y devuelve sus `Process`, en el mismo orden.

Salida esperada:

```
Padre: proceso de la tubería con PID 4501
Padre: proceso de la tubería con PID 4502
Padre: proceso de la tubería con PID 4503
Padre: la suma de los cuadrados del 1 al 10 es 385
```

Piensa.

1. Quita la etapa `Cuadrado`. ¿Qué sale? Compruébalo a mano.
2. ¿Puede `Sumar` escribir el resultado antes de que termine `GeneradorNumeros`? ¿Por qué?

Ejercicio 10. Varios hijos a la vez, con tiempo límite

`Ej10TodosALaVez` · Hijo: `Dormilon` · Completas `esperarConLimite` (6 líneas) y `dormirTodos` (9 líneas) · Modelos: ejemplos 8.1b y 9.1

Cuatro hijos que duermen 3, 1, 2 y 30 segundos. Para que trabajen a la vez, el padre los lanza **todos antes de esperar a ninguno**. Después espera a cada uno como mucho `LIMITE` segundos (5): el de 30 no llega y hay que terminarlo. `lanzar` y `leerSegundos` ya están hechos.

Pista. `waitFor(LIMITE, TimeUnit.SECONDS)` devuelve `true` si el hijo ha terminado a tiempo, y entonces `exitValue()` da su código de salida. Para lanzarlos todos antes de esperar, guarda sus `Process` en una lista: un bucle para lanzar y otro para esperar.

Salida esperada:

```
Hijo 4601: me duermo 3 s
Hijo 4604: me duermo 30 s
Hijo 4603: me duermo 2 s
Hijo 4602: me duermo 1 s
Hijo 4602: me despierto
Hijo 4603: me despierto
Hijo 4601: me despierto
Padre: el hijo 4604 no ha terminado a tiempo; lo termino
Padre: los hijos han dormido 6 s en total y yo he tardado 8 s
```

Las cuatro primeras líneas pueden salir en otro orden: los cuatro hijos arrancan a la vez.

Piensa.

1. El padre tarda unos 8 segundos. ¿Por qué no 5, que es el límite, ni 36, que es lo que duermen todos juntos?
2. Lanza y espera a cada hijo dentro del mismo bucle, uno detrás de otro. ¿Cuánto tarda ahora? ¿Por qué?

6. Para la prueba

En la prueba de RA1 completarás métodos como estos, **en papel y sin IntelliJ**. Cuando acabes los diez ejercicios:

- Vuelve a escribir en papel, sin mirar, los métodos de los ejercicios 3, 5, 8 y 10, y compáralos con los que te funcionan.
- Comprueba que sabes:
  - [ ] Explicar qué hace cada línea que has escrito.
  - [ ] Qué método de `Process` usa el padre para leer la salida del hijo, para escribir en su entrada y para leer su salida de error, y por qué se llaman así.
  - [ ] Por qué un padre se puede quedar colgado esperando a su hijo (ejercicio 6).
  - [ ] Cuándo usar `waitFor()`, `waitFor(tiempo, unidad)` y `exitValue()`.
  - [ ] Lanzar varios hijos a la vez y esperarlos con un tiempo límite (ejercicio 10).